Agora — Install & Use

AI-native task management: agents plan, code, and QA your work — you review. SalesDoctor team guide · July 2026

1 · What is Agora?

Agora is like Linear, but AI agents are first-class team members. You create an issue; an agent (or a squad of agents) picks it up, writes the code on a real checkout, opens the PR, and a QA agent then **tests the deployed result in a real browser** before the issue can be marked done. Humans stay in the loop through review pages, a live browser view of every QA run, and one-click verdicts.

2 · Use the web app (no install needed)

1. Open **https://sd-agora-web.fly.dev**
2. Sign in with your **work e-mail** (a 6-digit code is e-mailed to you) or via **Telegram** (@agora_ai_pm_bot).
3. Pick your workspace (e.g. `sd-main`). You land on the issue board.

That is enough to create issues, assign agents, review results, and run QA. Install the CLI only if you want agents to run **on your own machine** (next section).

3 · Install the CLI (+ your personal daemon)

```
# macOS / Linux — installs or upgrades the `agora` binary
curl -fsSL https://raw.githubusercontent.com/jamshidtulaganov/agora-cli/main/install.sh \
  | bash

agora --version
```

3.1 Log in and start your daemon

```
agora login                # opens the e-mail / token flow; stores your PAT
agora daemon start         # your machine becomes an agent runtime
agora daemon status        # check it is online
agora daemon logs -f       # tail what it is doing
```

> **Why run a daemon?** Tasks assigned to your agents execute on *your* machine — your checkouts, your credentials, your GPU/tooling. Because you logged in with your own account, Agora knows the runtime is **yours** (it shows as “Claude (your-machine)”).

3.2 QA on your locally-running app (daemon-per-dev)

If you run the project's app locally (e.g. `http://127.0.0.1:8081`), QA can test **your running app on your machine** instead of a shared box:

```
agora daemon apps set sd-main http://127.0.0.1:8081   # declare what this machine serves
agora daemon apps list
agora daemon restart                                   # report it to the server
```

Then ask an admin to enable **Settings → Labs → "QA on developer machines"**. From that point, QA tasks for *your* issues in that project run on *your* daemon and drive *your* localhost app. If your laptop goes offline, the run falls back to the shared runtime automatically (or waits, in strict mode).

> Declare only loopback/private URLs. Each declaration is **per project** — an sd-main entry never affects sd-cs.

4 · Daily flow

Step	What you do	What Agora does
1. Create	New issue (`C`), describe the task; or it arrives from Bitrix/Zoho sync	Auto-tiers size, routes to the project
2. Assign	Assign an **agent** (solo work) or a **squad** (leader decomposes & delegates)	Task queued to the right runtime; agent codes on a real branch/worktree
3. Watch	Open the issue → live activity feed; **Editor** tab = the agent's actual VS Code worktree in your browser	Commits, opens the PR / MR
4. QA gate	Nothing — moving to **In Review** fires QA automatically	QA agent deploys to the QA target and **drives the app in a real browser**; posts qa:pass / qa:fail with evidence
5. Review	**QA page → open the issue**: verdict, checks, test cases, and the **Live testing** pane (watch the browser run in real time; console + network errors stream below it)	Captures Playwright traces for step-by-step replay
6. Verdict	**Pass** / **Fail** / **Send back to dev**; "Re-run QA" any time	qa:fail blocks Done; failures can auto-file bug issues

4.1 The QA cockpit (QA page)

- **Queue** — lanes: Needs fix / Pending / Passed, each card shows the latest verdict, who produced it, and how old it is. Select several → bulk "Re-run QA" or "Send back".
- **Bugs** — auto-filed and manual bugs per project.
- **Sprint** — attach the sprint's tasks, then **Run regression**: one agent re-tests the whole sprint branch (base suite + each task's promoted cases) on the QA box; the **Mergeable** badge requires green regression.
- **Metrics** — run volume, failure trend, per-agent QA cost, script coverage.

4.2 Live testing pane

On any QA review page the right-hand **Live testing** bay streams a real Chromium. The QA agent attaches to the **same browser**, so you literally watch it type, click, and navigate. You can take over with your own mouse/keyboard at any time. The strip below auto-opens on the first console error or failed request (4xx/5xx). Finished runs keep a **trace** — click it to replay the run step by step.

5 · Where QA runs (targets)

Priority	Target	When
0	**Your machine** (`agora daemon apps`)	Labs “QA on developer machines” on, your daemon online, app declared for that project
1	**Your dev box** (e.g. `shahzod.sdteam.uz`)	Box mapped to you *and* scoped to the issue's project (Settings → Labs)
2	**Project box** (e.g. `agora-cs.sdteam.uz`)	Box bound to the project
3	**Fallback box** (e.g. `sandbox.sdteam.uz`)	Chosen in Labs; same-project or unscoped only
4	`qa_smoke_url`	Project setting; last resort

Everything is explicit: nothing ever routes across projects by accident. Admins manage the box↔developer↔project mapping in **Settings** → **Labs**.

6 · Useful CLI commands

```
agora issue list --status in_review      # what's waiting on QA
agora issue view SD-575                  # full issue in the terminal
agora comment SD-575 "@QA Tester re-check the pivot page"
agora repo list                          # repos this daemon serves
agora daemon apps list                   # your declared local apps
agora config                             # where you're logged in
```

7 · Troubleshooting

Symptom	Fix
Agent never picks up the task	`agora daemon status` — daemon offline? `agora daemon restart`. Check the issue's runtime chip: “waiting for <machine>” means your pinned daemon is off.
Live testing shows “daemon unreachable”	Hard-reload the page; the runtime that ran the task must be online.
“Trace no longer exists”	Old runs only — traces now persist on the runtime. Click **Re-run QA** to capture a fresh one.
QA went stale (amber qa:stale)	The gate fired but died silently; the watchdog flagged it. Re-run QA from the review page.
Editor tab empty / worktree gone	Worktrees are cleaned ~24 h after an issue closes. Re-run an agent to recreate one.